

Collaborating on ML Without Handing Over Your Data: Privacy-Preserving Learning via Autoencoder Latent Embeddings

NeurIPS 2022: a framework that lets peers jointly train models while their raw features stay private

Picture a common situation: two companies each hold a different set of variables about the same users, one has transaction histories, the other credit records. If they could combine their features, both could predict a shared target, say, the probability of default, far more accurately. But the data is sensitive. Privacy policies and intellectual property (IP) laws typically forbid handing raw features to the other party, even over a perfectly secure channel. So the collaboration is called off, and the accuracy gain it promised is lost with it.

This NeurIPS 2022 paper from Los Andes University, Rappi, and Amazon offers a direct answer: if you cannot share the raw data, share an obfuscated representation of it instead. Each peer first passes its tabular data through an autoencoder to produce a latent-space representation, and only that encoded embedding is shared. The peers then join their embeddings on a common observation ID and train a downstream supervised model on the combined result. The original features never leave the party that owns them.

The authors validate the framework on three public benchmarks. The headline finding is that replacing raw features with shared latent embeddings costs less than 10 percentage points (pp) of downstream performance, while the autoencoder's reconstruction error stays between 5% and 11%, so the sensitive features are hidden yet the predictive power is preserved almost intact.

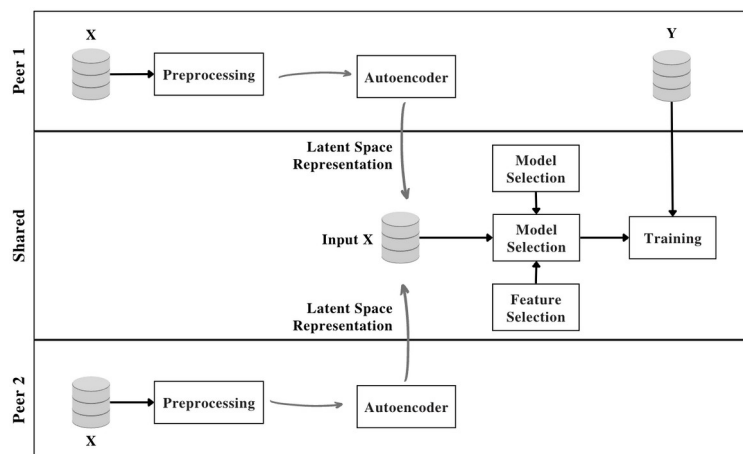


Figure 1. The privacy-preserving machine learning framework. Each of the two peers encodes its database into a latent-space representation and shares only that representation with a common data source, which then drives model selection, feature selection, and training. The raw features X never leave either peer.

Paper: <https://arxiv.org/abs/2211.05717>

Why existing privacy tools fall short

Academia and industry have built several privacy-preserving strategies, but each comes with a real limitation. Encryption approaches such as homomorphic encryption offer strong security, yet they are hard to deploy in practice because of their technology requirements. Federated learning decentralizes training across devices, but it assumes every peer holds the same kind of information, so it cannot handle two peers that each contribute different features. Differential privacy masks individual values by adding noise, which can significantly reduce data utility. And linear dimensionality reduction such as Principal Component Analysis (PCA) obfuscates features but may discard important nonlinear relationships between them.

Most of these methods only protect data in transit, distort individual observations, or carry heavy maintenance costs. The gap they leave open is what this paper targets: can deep representation learning encode data into a form that is safe to share yet still carries its predictive structure?

Method: one extra encoding step in the sharing pipeline

The elegance of the framework is that it does not rewrite the usual data-sharing pipeline; it simply inserts one step. Instead of merging raw datasets, each peer first passes its features through an autoencoder to obtain a latent-space representation, an obfuscated version of the data that is ready to be shared. The peers then align and join these representations using a shared observation ID and train a single supervised model on the combined embeddings to predict the same target.

So that the conclusions reflect the framework rather than a lucky round of network tuning, the authors fix the same autoencoder architecture across every experiment. The encoder has four layers, compressing the N input features through hidden sizes 128, 64, and 40 down to an embedding of size M ; the decoder mirrors this back up to N . Every layer uses ReLU, the loss is Mean Absolute Error (MAE), and training uses the Adam optimizer with a learning rate fixed at 0.0001.

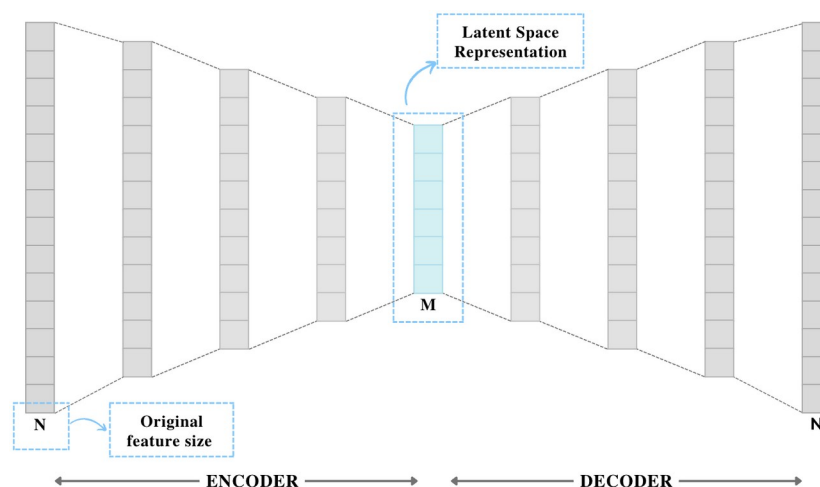


Figure 3. The autoencoder structure. The encoder compresses the original feature dimension N down to a latent dimension M , and the decoder reconstructs M back to N . The middle vector of size M is the latent-space representation that gets shared.

Five scenarios, one design choice at a time

The experiments are organized as an ablation across five scenarios, so each design decision can be seen in isolation. Scenario 0 is the raw-data baseline with no privacy protection; its performance is the bar the later scenarios try to hold. Scenario 1 uses a single shared autoencoder to encode one combined dataset. Scenario 2 simulates genuine two-peer collaboration: each peer has its own autoencoder, and the two embeddings are then joined.

Scenarios 3 and 4 are the non-naive versions of Scenarios 1 and 2. Non-naive means the autoencoder becomes a multitask network that also predicts the downstream target while encoding, so the latent space is steered by the task rather than by reconstruction alone. Figure 2 traces, in three colors, how data flows under the different scenarios.

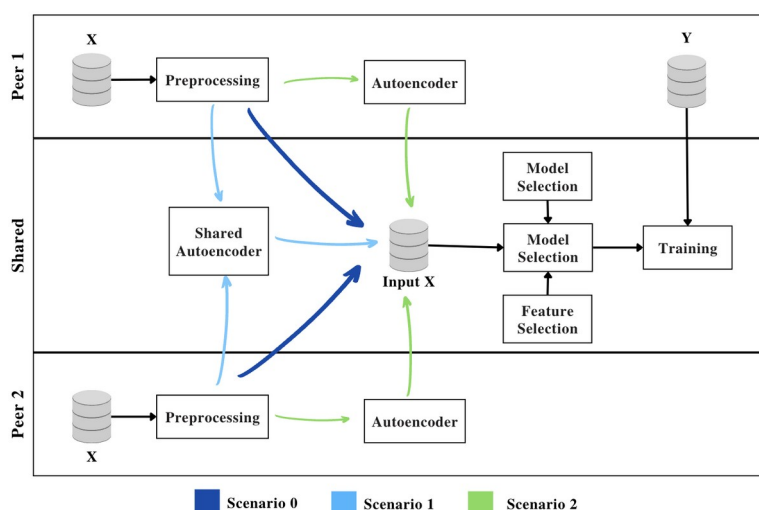


Figure 2. The general autoencoder structure for the training phase and the scenarios explored in this work. Dark blue is Scenario 0 (raw baseline), light blue is Scenario 1 (a single shared autoencoder), and green is Scenario 2 (an individual autoencoder per peer). The non-naive Scenarios 3 and 4 add target prediction on top.

Three benchmark datasets

To test the framework under realistic and varied conditions, the authors chose three public datasets that deliberately differ in downstream task, dimensionality, and feature type, which lets them probe how robust and scalable the encoding approach is. All experiments ran on a modest setup: a two-core 2.30 GHz Intel Xeon CPU, an Nvidia K80 or T4 GPU with 12 GB of memory, and 12 GB of RAM.

Table 1. The three public benchmark datasets and their downstream tasks.

Dataset	Observations	Features	ML task
House Pricing	21,613	12	Regression
MNIST Numbers	35,000	784	Ten-class classification
Buzz in Social Media	87,488	77	Regression

Results: hide the raw features, keep the performance

The central result is that swapping raw features for encoded representations barely hurts predictive performance. On House Pricing, the raw-data baseline reaches a test R^2 of about 90.29%, and Scenario 2, which simulates two peers each with their own autoencoder, still reaches 89.33%, an almost negligible drop given that the data has been obfuscated. On Buzz in Social Media, the baseline R^2 is around 96.19% while the encoded scenarios hold in the high eighties to low nineties, and for MNIST digit classification the test accuracy falls only from a 92% baseline into the 84%–88% band under encoding.

The two non-naive scenarios are the most telling. Once the autoencoder is steered by the target variable, they recover much of the lost ground: on Buzz, test R^2 climbs from 91.55% (Scenario 1) to 94.03% (Scenario 3), and on MNIST, accuracy returns from 88% to 91%. Shaping the latent space with the downstream task clearly pays off. The table below summarizes the test-set numbers across all three datasets.

Table 2. Test-set performance of the three datasets across the five scenarios (House Pricing and Buzz report R^2 ; MNIST reports accuracy).

Scenario	House Pricing (R^2)	MNIST (Accuracy)	Buzz (R^2)
Scenario 0: raw baseline	90.29%	92%	96.19%
Scenario 1: single shared AE	84.27%	88%	91.55%
Scenario 2: individual AEs	89.33%	84%	89.01%
Scenario 3: non-naive shared	89.21%	91%	94.03%
Scenario 4: non-naive individual	87.36%	84%	87.90%

The numbers worth remembering

A few numbers capture the impact. Across all use cases, swapping raw features for shared latent representations dropped performance by less than 10 percentage points, with reconstruction error from 5% to 11%. On House Pricing, test R^2 fell only from 90.29% to 89.33% with two individual autoencoders; on Buzz, the non-naive shared autoencoder (Scenario 3) reached 94.03% against a 96.19% baseline. The shared autoencoders correctly estimated 96% to 98% of observations per feature (MAPE of 5% or less), confirming the embeddings retain the core structure of the data.

Takeaway and what comes next

The takeaway is simple: rather than exchanging raw sensitive data, collaborating organizations can share the latent-space embeddings produced by autoencoders. This keeps the original features private while preserving most of the predictive power, dropping by less than 10 percentage points across three benchmarks, and making the autoencoder task-aware narrows the gap even further.

The authors point to clear next steps: autoencoders tailored to each dataset, and formal measures of privacy strength that quantify how well the encoding resists reconstruction. For teams that want cross-organization collaboration, it is a pragmatic, low-barrier place to start.